

I-Chip PS 1 Documentation

Yash Dubey

February 2026

Contents

1 Architectural Approach	1
2 Resource Usage Optimization	1
3 Handling Multiple Channels	2
4 Blur Technique	2
5 State Machine	4
6 Python Scripts For Conversion	5

1 Architectural Approach

The architecture is partitioned into three functional modules: a Coordinate Generator (coordgen), a Non-Restoring Division Unit(div), and a Central FSM(top).

Coordinate Generation: The coordgen block accepts target dimensions (H_{out}, W_{out}) as input and generates the corresponding output coordinates (x_{out}, y_{out}). It is managed via specialized control signals—incx, incy, rstx, and rsty—for precise incrementing and resetting of the coordinate counters.

Division Unit: It implements a non-restoring division algorithm to compute scaling ratios. This implementation ensures an $O(k)$ time complexity, where k represents the maximum bit-width of the operands, providing deterministic performance for hardware execution.

Processing Pipeline: Designed for image resizing, the system supports both upscaling and downscaling. During downscaling, the design integrates a pre-blurring stage prior to bilinear interpolation. This specific sequence mitigates aliasing artifacts and enhances the Structural Similarity Index (SSIM) of the output.

2 Resource Usage Optimization

To optimize resource utilization, the interpolation architecture employs a serialized "accumulation and store" methodology. The process begins by mapping output coordinates to their corresponding input pixels and generating the necessary fractional weights. The system then sequentially fetches neighboring pixel intensities, multiplying them by their respective weights and summing the results across four accumulation states (acc1 through acc4). Upon completion, the design transitions to a write state, where values undergo rounding before being committed to the output image array. Boundary conditions are managed via zero-padding, effectively treating any out-of-bounds pixel coordinates as having zero intensity.

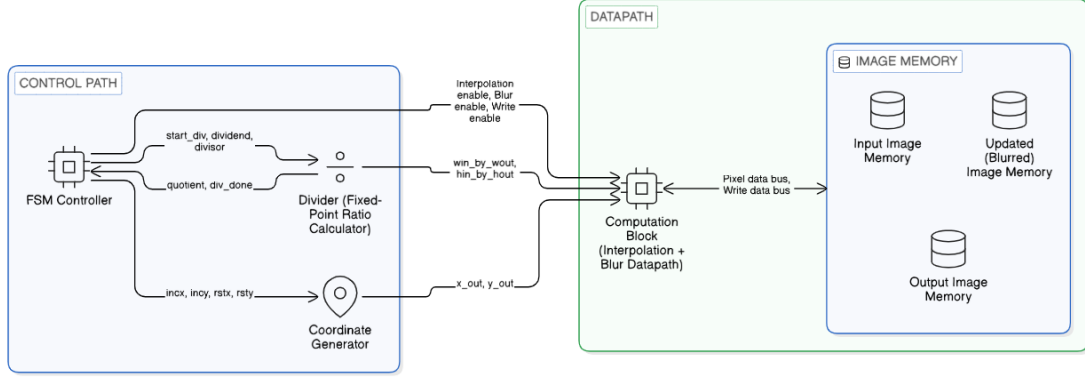


Figure 1: High Level Block Diagram of Design

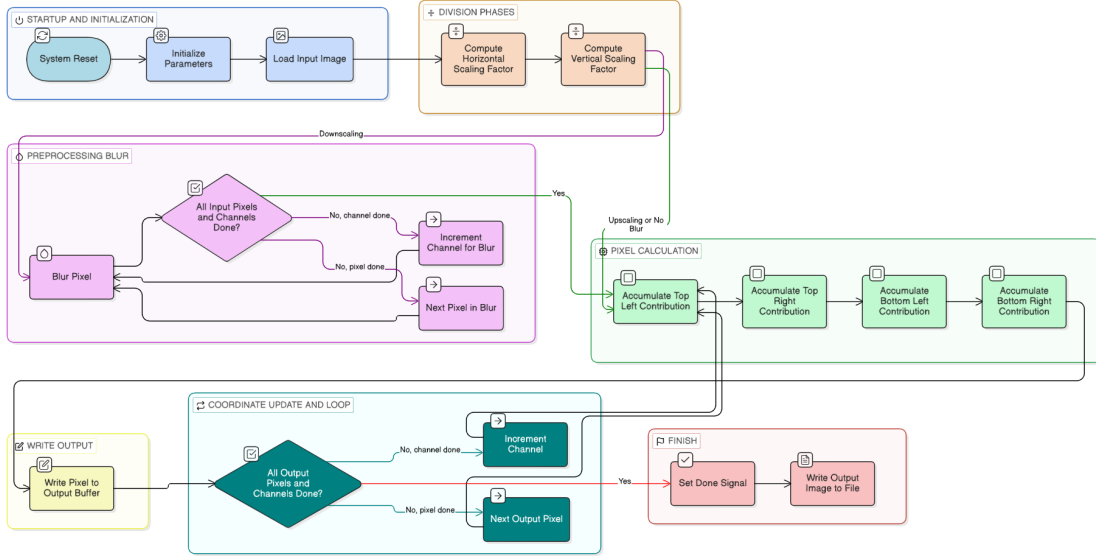


Figure 2: State Flow of Hardware Image Rescaling Engine From Initialization and Scaling to Adaptive Blur and Bilinear Accumulation.

3 Handling Multiple Channels

In grayscale mode, the system processes the image and writes the final results to output memory upon completion. For RGB images, the data is organized in a planar format ($RRR...GGG...BBB$), where each color channel is stored in a separate memory plane. The processing pipeline executes bilinear interpolation sequentially on the Red, Green, and Blue channels, followed by a channel-wise blurring operation.

4 Blur Technique

This implementation uses a Gaussian bell curve to weight the center pixel more heavily than its neighbors using the kernel given below, creating a smoother transition than the uniform approach of a box blur. By eliminating the mathematical "secondary lobes" that cause blocky artifacts and ringing, this method acts as a superior low-pass filter. The result is a natural-looking blur that cleans up high-frequency noise without introducing artificial patterns.

$$\begin{bmatrix} 16 & 32 & 16 \\ 32 & 64 & 32 \\ 16 & 32 & 16 \end{bmatrix}$$

By utilizing bit-shifted values for its elements, this matrix optimizes performance by replacing hardware-intensive multiplication with simple bitwise operations for weight computation.

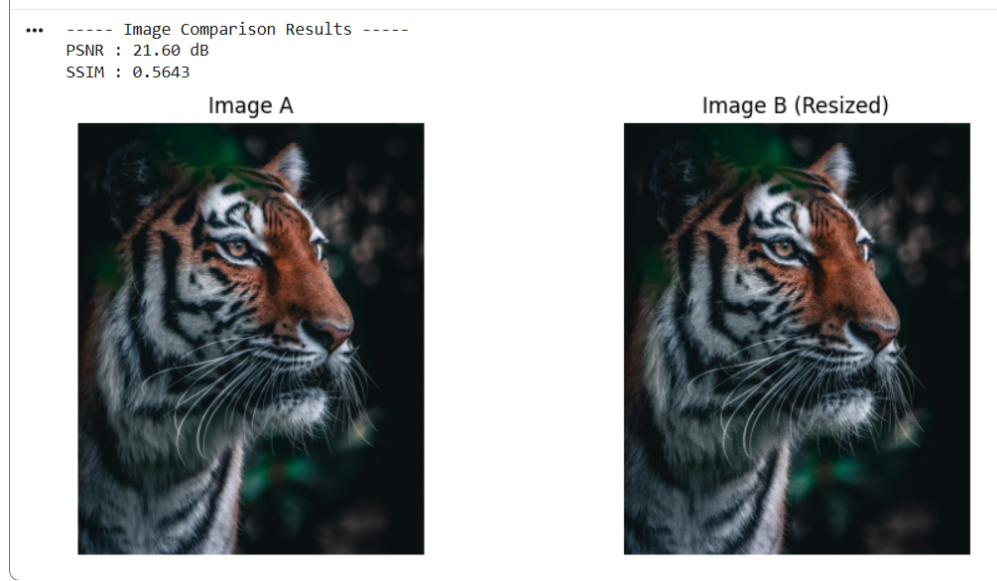


Figure 3: SSIM before Blur

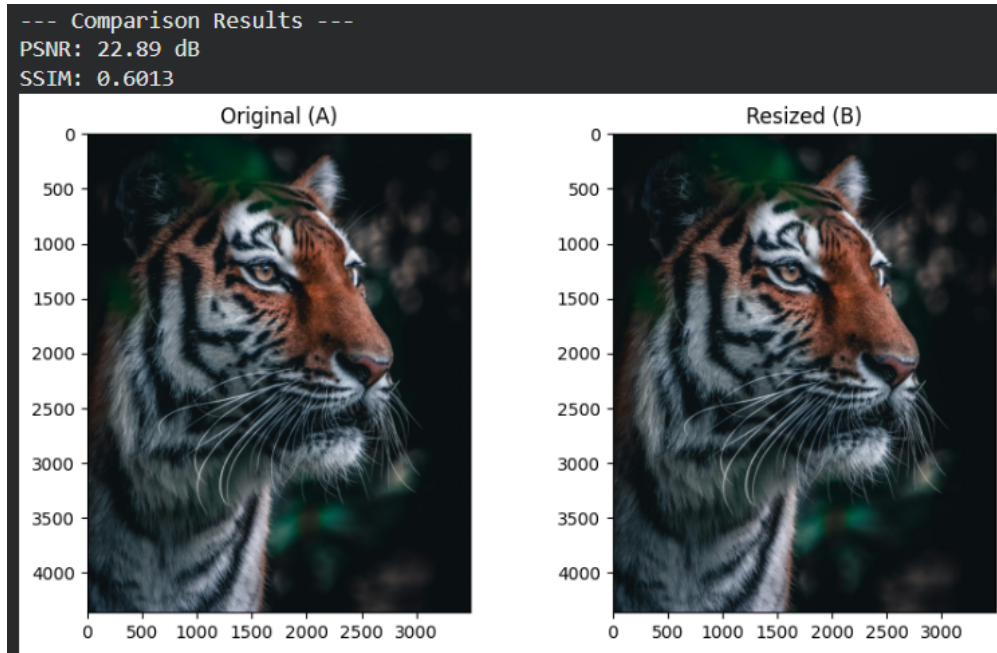


Figure 4: SSIM after Blur

5 State Machine

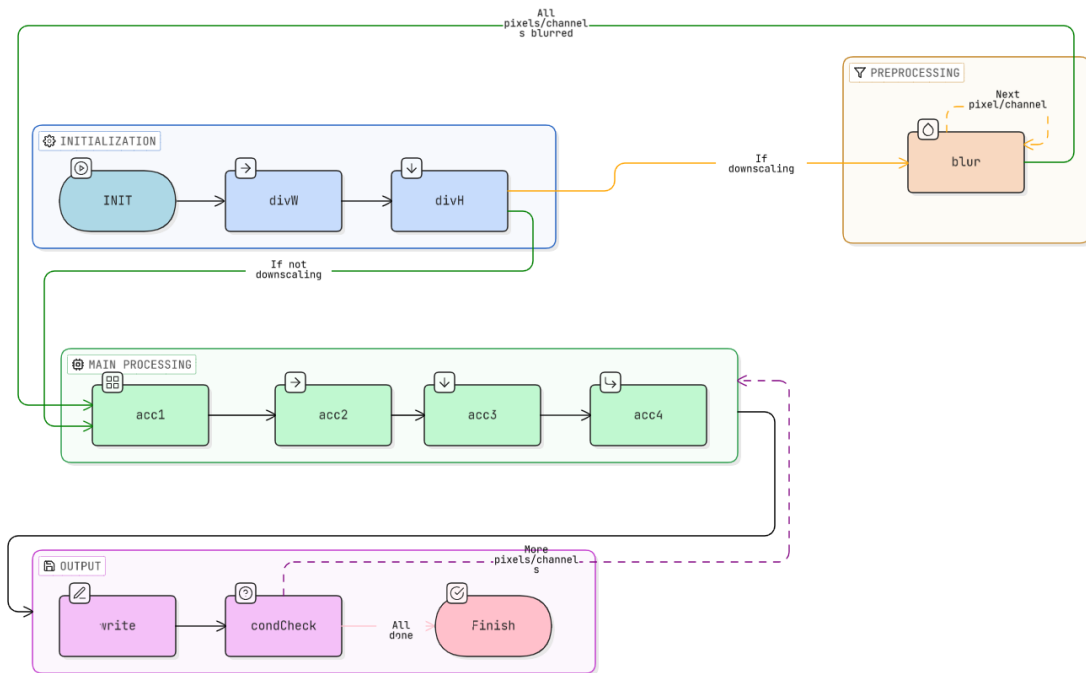


Figure 5: State Diagram

This self-explanatory diagram outlines the specific state transitions the machine follows to produce the final image.

6 Python Scripts For Conversion

```
1 from PIL import Image
2 import numpy as np
3 # Greyscale to hex
4 # Configuration
5 INPUT_IMG = "image.jpg"
6 OUTPUT_HEX = "image.hex"
7
8 # 1. Load and convert to Grayscale ('L' mode: 8-bit pixels, black and white)
9 img = Image.open(INPUT_IMG).convert('L')
10 img_np = np.asarray(img, dtype=np.uint8)
11
12 H, W = img_np.shape
13
14 # 2. Write to Hex file
15 # We flatten the array to 1D to make writing much faster
16 with open(OUTPUT_HEX, "w") as f:
17     # This creates a list of hex strings and joins them with newlines
18     hex_data = "\n".join(f"{val:02x}" for val in img_np.flatten())
19     f.write(hex_data + "\n")
20
21 # 3. Output Verilog Parameters
22 print(f"Success! Saved to {OUTPUT_HEX}")
23 print("-" * 20)
24 print("Verilog Parameters:")
25 print(f"parameter W_IN = {W};")
26 print(f"parameter H_IN = {H};")
27 print(f"parameter CH   = 1;")
```

Listing 1: Greyscale to Hex Conversion

```

1 from PIL import Image
2 import numpy as np
3 # image to RRRGGGBBB
4 # Configuration
5 INPUT_IMG = "image.jpg"
6 OUTPUT_HEX = "image.hex"
7
8 # 1. Load image and force RGB
9 img = Image.open(INPUT_IMG).convert('RGB')
10 img_np = np.asarray(img, dtype=np.uint8)
11
12 H, W, CH = img_np.shape # CH = 3
13
14 # 2. Extract planes
15 R = img_np[:, :, 0].flatten()
16 G = img_np[:, :, 1].flatten()
17 B = img_np[:, :, 2].flatten()
18
19 # 3. Concatenate planes: R | G | B
20 planar_data = np.concatenate([R, G, B])
21
22 # 4. Write HEX file (1 byte per line)
23 with open(OUTPUT_HEX, "w") as f:
24     hex_data = "\n".join(f"{val:02x}" for val in planar_data)
25     f.write(hex_data + "\n")
26
27 # 5. Verilog parameters
28 print(f"Success! Saved to {OUTPUT_HEX}")
29 print("-" * 30)
30 print("Verilog Parameters:")
31 print(f"parameter W_IN = {W};")
32 print(f"parameter H_IN = {H};")
33 print(f"parameter CH = 3;")
34 print(f"parameter PIXEL_BITS = 8;")
35 print(f"parameter PLANE_SIZE = W_IN * H_IN;")

```

Listing 2: Image To RRR...GGG...BBB... Conversion

```

1 from PIL import Image
2 import numpy as np
3 # RRRGGG BBB to image
4 # Configuration
5 INPUT_HEX = "image.hex"
6 OUTPUT_IMG = "image.jpg"
7
8 # ---- MUST match encoder parameters ----
9 W = 872 # width
10 H = 1090 # height
11 CH = 3
12 # -----
13
14 PLANE_SIZE = W * H
15
16 # 1. Read hex file
17 with open(INPUT_HEX, "r") as f:
18     hex_lines = f.read().split()
19
20 # Convert hex strings to uint8
21 data = np.array([int(x, 16) for x in hex_lines], dtype=np.uint8)
22
23 # 2. Sanity check
24 expected_len = PLANE_SIZE * CH
25 assert len(data) == expected_len, \
26     f"Hex size mismatch: expected {expected_len}, got {len(data)}"
27
28 # 3. Split planar data
29 R = data[0:PLANE_SIZE].reshape(H, W)
30 G = data[PLANE_SIZE:2*PLANE_SIZE].reshape(H, W)
31 B = data[2*PLANE_SIZE:3*PLANE_SIZE].reshape(H, W)
32
33 # 4. Reconstruct RGB image
34 img_np = np.stack([R, G, B], axis=2)
35
36 # 5. Save image
37 img = Image.fromarray(img_np, mode="RGB")
38 img.save(OUTPUT_IMG)
39
40 print("Decode successful!")
41 print(f"Saved image as {OUTPUT_IMG}")

```

Listing 3: RRR...GGG...BBB to Image Convertor